

MINISTRY OF EDUCATION, CULTURE AND RESEARCH OF THE REPUBLIC OF MOLDOVA

Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software and Automation Engineering

Oberst Eduard, FAF-243

Report

Laboratory work nr. 4

The xv6 Shell from the Outside

Course: Operating Systems (Track B: xv6)

Checked by:
Patricia Reitman, *University Assistant*
Department of Software and Automation Engineering,
FCIM Faculty, UTM

Chişinău – 2026

Purpose of the work

The purpose of this laboratory work is to understand what a command shell must do and what bash adds on top of that, by using the xv6 shell (`user/sh.c`, about 500 lines) side by side with bash. A small program that prints its own `argv` shows exactly who expands `*` and who removes the quotes: the program, the kernel or the shell. Before that, the `sleep` program from Lab 2 is brought up to date with the current xv6, where the system call `sleep()` was renamed to `pause()`.

Preparation: xv6 builds and boots

```
~$ qemu-system-riscv64 --version | head -1
QEMU emulator version 10.2.1 (Debian 1:10.2.1+ds-1ubuntu3.2)
~$ riscv64-linux-gnu-gcc --version | head -1
riscv64-linux-gnu-gcc (Ubuntu 15.2.0-16ubuntu1) 15.2.0
~$ cd ~/xv6-riscv && make qemu
xv6 kernel is booting


hart 1 starting
hart 2 starting
init: starting sh
$
```

The work was done on Ubuntu 26.04.1 LTS with xv6-riscv at revision `06aad25`. The RISC-V compiler and QEMU were already installed, and xv6 builds and boots to the `$` prompt (exit with Ctrl-A, then X).

Part 1. Fix sleep for the current xv6

Interesting Command Output:

```
~/xv6-riscv$ grep -n "pause\|sleep" user/user.h
25:int pause(int);
```

Figure 1.1 – The grep result in user.h

The tree declares only `pause`, so the program has to call `pause()` instead of `sleep()`. `user/sleep.c` was written as follows, and `$U/_sleep\` was added to UPROGS in the Makefile (each line starts with a TAB and ends with a backslash, with nothing after it).

```
#include "kernel/types.h"
#include "user/user.h"


int
main(int argc, char *argv[])
{
  if(argc != 2){
    fprintf(2, "usage: sleep <ticks>\n");
    exit(1);
  }
  pause(atoi(argv[1]));   // was sleep() before Aug 2025
  exit(0);
}
```

Figure 1.2 – The updated sleep.c

```
~/xv6-riscv$ make qemu
...
init: starting sh
$ sleep 20
$ sleep
usage: sleep <ticks>
$ sleep 10 20
usage: sleep <ticks>
```

Figure 1.3 – sleep running in xv6, with and without an argument

`sleep 20` waits 20 timer ticks (about two seconds) and returns to the prompt without output. Without an argument, or with too many, the program prints the usage line on standard error (file descriptor 2) and exits with status 1.

Part 2. bash versus the xv6 shell

Every command was run twice: in bash, in an empty test folder that contains a file named README (`mkdir ~/shtest && cd ~/shtest && touch README`), and at the xv6 `$` prompt.

Interesting Command Output:

```
~/shtest$ echo hello > f ; cat < f
hello
~/shtest$ ls | grep READ
README
~/shtest$ (echo a; echo b) | wc
      2       2       4
~/shtest$ echo one ; echo two
one
two
~/shtest$ echo bg &
[1] 39185
bg
[1]+  Done                    echo bg
~/shtest$ ls *
README  f
~/shtest$ echo $HOME
/home/capziro
~/shtest$ echo "a b"
a b
~/shtest$ mkdir d ; cd d ; ls ..
README  d  f
```

Figure 2.1 – The commands run in bash

```
$ echo hello > f ; cat < f
hello
$ ls | grep READ
README          2 2 2441
$ (echo a; echo b) | wc
2 2 4
$ echo one ; echo two
one
two
$ echo bg &
bg
$ ls *
ls: cannot open *
$ echo $HOME
$HOME
$ echo "a b"
"a b"
$ mkdir d ; cd d ; ls ..
exec cd failed
.               1 1 1024
..              1 1 1024
README          2 2 2441
cat             2 3 36728
...
d               1 27 32
$ echo hi
hi
$ ^[[A
exec ^[[A failed
$ ec<Tab>
exec ec failed
```

Figure 2.2 – The same commands run in xv6

Table 2.1 – Comparison of bash and the xv6 shell

Command	**Tests**	**bash**	**xv6**
`echo hello > f ; cat < f`	redirection	hello	hello – the same: `<` and `>` are supported.
`ls | grep READ`	pipe	README	`README 2 2 2441` – works; xv6 `ls` always prints type, inode and size.
`(echo a; echo b) | wc`	grouping, list	2 2 4	2 2 4 – the same: `( )` and `;` are supported.
`echo one ; echo two`	sequence	one / two	one / two – the same.
`echo bg &`	background	`[1] PID`, bg, then `[1]+ Done`	bg – runs in the background, but there are no job numbers, no `jobs`, no Done message.
`ls *`	globbing	README f	`ls: cannot open *` – no globbing, ls gets a literal `*`.
`echo $HOME`	variables	/home/capziro	`$HOME` – no variables, printed literally.
`echo "a b"`	quotes	a b	`"a b"` – quotes are not removed; echo gets two words `"a` and `b"`.
`mkdir d ; cd d ; ls ..`	PATH	README d f	`exec cd failed`, then the listing of `/`: `cd` is built-in only at the start of a line. On separate lines, `cd d` works and then `ls` fails: `exec ls failed` (no PATH).
arrow up, Tab, Ctrl+R	history, completion	recall, complete, search the history	nothing: the raw bytes `^[[A`, a TAB or `^R` go into the line and the command fails with `exec … failed`.

```
$ cd d
$ ls
exec ls failed
$ ls ..
exec ls failed
$ /ls ..
.              1 1 1024
..             1 1 1024
README         2 2 2441
...
d              1 27 32
$ /echo still works
still works
$ cd /
$ ls README
README         2 2 2441
```

Figure 2.3 – ls failing after cd d, and the working alternative

Observations:

1. **List the features both shells have, and the ones only bash has.**

   Both shells: running a program with arguments, input/output redirection with `<`, `>` and `>>`, pipes `|`, sequences with `;`, grouping with `( )`, background execution with `&`, and the built-in `cd`.

   Only bash: globbing (`*`, `?`, `[ ]`), variables and `$` expansion, quote removal (`" "`, `' '`, `\`), searching programs in PATH, command history (arrow up, Ctrl+R), Tab completion, job control (`jobs`, `kill %1`, `fg`), the operators `&&` and `||`, command substitution `$( )`, brace expansion `{1..5}`, arithmetic `$(( ))`, redirection of stderr (`2>`, `2>&1`), and a full scripting language (if, for, functions).

2. **In xv6, ls stops working after cd d. Why? Find a way to run ls from inside d anyway.**

   The xv6 shell has no PATH: it calls `exec(argv[0], argv)` with the name exactly as typed, so `ls` is looked up relative to the current directory. All xv6 programs live in the root directory `/` (they are copied there by `mkfs` from UPROGS). After `cd d`, `ls` means `d/ls`, which does not exist, so the shell prints `exec ls failed`. The fix is to use an absolute path: `/ls` or `/ls ..` works from any directory (or go back with `cd /`).

   A second detail: in `mkdir d ; cd d ; ls ..` the `cd` is not at the start of the line, so the shell does not recognise it as a built-in and tries to run a program called `cd` – `exec cd failed` – and the directory does not change. That is why `cd d` has to be typed on its own line.

3. **Which missing feature would you miss most in daily work, and which would be easiest to add?**

   Most missed: history and Tab completion together with globbing – without them every long command or file name must be typed in full, and `*` cannot be used at all. Easiest to add: a PATH fallback. In the `EXEC` case of `runcmd()`, if `exec(argv[0])` fails and the name does not start with `/`, the shell can build the string `"/" + argv[0]` and call `exec` again – only a few lines of code, and `ls` would work from any directory.

Part 3. Who expands *? A program that prints its argv

Interesting Command Output:

```
#include "kernel/types.h"
#include "user/user.h"


int
main(int argc, char *argv[])
{
  printf("argc = %d\n", argc);
  for(int i = 0; i < argc; i++)
    printf("argv[%d] = [%s]\n", i, argv[i]);
  exit(0);
}


~/xv6-riscv$ grep -n "_sync\|_sleep\|_args" Makefile
152:    $U/_sync\
153:    $U/_sleep\
154:    $U/_args\
```

Figure 3.1 – args.c and the UPROGS line

```
$ args one   two   three
argc = 4
argv[0] = [args]
argv[1] = [one]
argv[2] = [two]
argv[3] = [three]
$ args *.c "x y"
argc = 4
argv[0] = [args]
argv[1] = [*.c]
argv[2] = ["x]
argv[3] = [y"]
$ args $HOME
argc = 2
argv[0] = [args]
argv[1] = [$HOME]
$ args 'single quoted'
argc = 3
argv[0] = [args]
argv[1] = ['single]
argv[2] = [quoted']
```

Figure 3.2 – args output in xv6

For Linux the same program was written as `args_linux.c` (with `<stdio.h>` and `return 0`) and compiled with `gcc -o args args_linux.c`. It was run in a folder that contains some `.c` files.

```
#include <stdio.h>


int
main(int argc, char *argv[])
{
  printf("argc = %d\n", argc);
  for(int i = 0; i < argc; i++)
    printf("argv[%d] = [%s]\n", i, argv[i]);
  return 0;
}


~/os-lab4$ gcc -o args args_linux.c
~/os-lab4$ ls
args  args_linux.c  hello.c  world.c
~/os-lab4$ ./args one   two   three
argc = 4
argv[0] = [./args]
argv[1] = [one]
argv[2] = [two]
argv[3] = [three]
~/os-lab4$ ./args *.c "x y"
argc = 5
argv[0] = [./args]
argv[1] = [args_linux.c]
argv[2] = [hello.c]
argv[3] = [world.c]
argv[4] = [x y]
~/os-lab4$ ./args $HOME
argc = 2
argv[0] = [./args]
argv[1] = [/home/capziro]
~/os-lab4$ ./args 'single quoted'
argc = 2
argv[0] = [./args]
argv[1] = [single quoted]
```

Figure 3.3 – args_linux output in bash

Observations:

1. **Paste both outputs for args *.c "x y". How many arguments did each program receive?**

   Both outputs are shown in Figures 3.2 and 3.3.

   - xv6: `argc = 4` – `[args] [*.c] ["x] [y"]`. The pattern stayed `*.c`, and the quoted text was split at the space, with the quote characters still inside the words.
   - bash: `argc = 5` – `[./args] [args_linux.c] [hello.c] [world.c] [x y]`. `*.c` became the three file names, and `"x y"` became one argument `x y` without quotes.

2. **So who turns *.c into file names and removes the quotes: the program, the kernel, or the shell?**

   The shell. The program is identical in both systems and only prints what it receives; the kernel's `exec` copies the `argv` array unchanged into the new process. The only difference is the shell: bash performs globbing, variable expansion and quote removal before calling `exec`, while the xv6 shell only splits the line at whitespace and passes the words on as they are.

3. **What happened to the extra spaces in one two three?**

   They disappeared in both shells. Spaces are separators between words, not part of any argument; several spaces in a row count as one separator. The program received exactly `one`, `two` and `three`. To keep the spaces in bash they would have to be quoted (`"one  two"`), which xv6 cannot do.

Part 4. Read the shell

Interesting Command Output:

```
~/xv6-riscv$ wc -l user/sh.c
499 user/sh.c
~/xv6-riscv$ grep -n "#define" user/sh.c
8:#define EXEC  1
9:#define REDIR 2
10:#define PIPE  3
11:#define LIST  4
12:#define BACK  5
14:#define MAXARGS 10
```

Figure 4.1 – The command kinds defined in sh.c

```
~/xv6-riscv$ grep -n "case " user/sh.c
75:  case EXEC:
83:  case REDIR:
93:  case LIST:
101:  case PIPE:
125:  case BACK:
284:  case 0:
286:  case '|':
287:  case '(':
288:  case ')':
289:  case ';':
290:  case '&':
291:  case '<':
294:  case '>':
391:    case '<':
394:    case '>':
397:    case '+': // >>
469:  case EXEC:
475:  case REDIR:
481:  case PIPE:
487:  case LIST:
493:  case BACK:
```

Figure 4.2 – Where each kind is run

Lines 75–125 are the `switch` in `runcmd()`, which executes each kind of command; lines 284–297 are in `gettoken()`, which recognises the special characters; lines 391–397 parse redirections, and lines 469–493 are in `nulterminate()`.

```
~/xv6-riscv$ grep -n "cd" user/sh.c | head
170:        fprintf(2, "cannot cd %s\n", cmd + 3);
~/xv6-riscv$ sed -n '158,176p' user/sh.c
  // Read and run input commands.
  while (getcmd(buf, sizeof(buf)) >= 0) {
    char *cmd = buf;
    while (*cmd == ' ' || *cmd == '\t')
      cmd++;
    if (*cmd == '\n') // is a blank command
      continue;
    if (cmd[0] == 'c' && cmd[1] == 'd' && cmd[2] == ' ') {
      // Chdir must be called by the parent, not the child.
      cmd[strlen(cmd) - 1] = 0; // chop \n
      if (chdir(cmd + 3) < 0)
        fprintf(2, "cannot cd %s\n", cmd + 3);
    } else {
      if (fork1() == 0)
        runcmd(parsecmd(cmd));
      wait(0);
    }
  }
```

Figure 4.3 – The code that handles cd

Observations:

1. **Name the five kinds of command the xv6 shell knows and the character that produces each.**
   - `EXEC` (1) – a plain command: a program name and its arguments, no special character;
   - `REDIR` (2) – redirection, produced by `<`, `>` or `>>`;
   - `PIPE` (3) – a pipeline, produced by `|`;
   - `LIST` (4) – a sequence, produced by `;`;
   - `BACK` (5) – a background command, produced by `&`.
   - Parentheses `( )` only group commands (parsed by `parseblock`); they do not form a sixth kind.
2. **Where is cd handled, and why is it not run as a separate program?**

   In `main()` of `user/sh.c` (lines 165–170): if the line starts with the three characters `cd `, the shell itself calls `chdir(cmd + 3)` and does not fork. All other commands are run in a child created by `fork1()`. The current directory belongs to each process, so a `cd` program running in a child would change only the child's directory, which is lost when the child exits – as the comment says, "Chdir must be called by the parent, not the child". Because the test only looks at the beginning of the line, `cd` inside `a ; cd d` is not recognised (Part 2).
3. **Search sh.c for any code that handles * or $. What does that explain about Part 2?**

   There is none. The tokenizer knows only whitespace (`" \t\r\n\v"`) and the symbols `<|>&;()`; it has no code for `*`, `?`, `$` or quotes, and no PATH or history. A word is passed to `exec` exactly as typed. This explains Part 2: `ls *` gave `ls: cannot open *`, `echo $HOME` printed `$HOME`, `echo "a b"` kept the quotes, and `args *.c "x y"` received `*.c`, `"x` and `y"`.

Conclusion

In this laboratory work I compared the xv6 shell with bash and found out by experiment what a shell really has to do. First I updated my `sleep` program for the current xv6, where the system call is now called `pause()`, added it to UPROGS and checked that it waits and prints its usage message correctly.

Running the same commands in both shells showed that the essential core is small: reading a line, splitting it into words, `fork` + `exec`, redirection with `<` and `>`, pipes, `;`, `( )`, `&` and a built-in `cd`. The xv6 shell does all of this in about 500 lines. Everything else that makes bash comfortable – globbing, variables, quote removal, PATH search, history, completion and job control – is added on top. Without PATH, `ls` stops working after `cd d` and must be called as `/ls`; I also noticed that xv6 recognises `cd` only at the beginning of a line.

The `args` program answered the main question of the lab: the program and the kernel pass the arguments through unchanged, and it is the shell that expands `*.c` into file names, expands `$HOME` and removes the quotes. Reading `user/sh.c` confirmed this: it defines only five kinds of command (EXEC, REDIR, PIPE, LIST, BACK), handles `cd` in the parent process because a child cannot change the parent's directory, and has no code at all for `*` or `$`.